CardioIA – Fase 3 · Parte 1

Armazenamento e processamento local (Edge Computing) no ESP32

1. Visão geral

Esta parte simula, no Wokwi, o "coração" de um **wearable cardíaco** baseado em ESP32. O dispositivo coleta sinais vitais, decide localmente se há risco (Edge Computing), guarda cada leitura em memória local e só envia os dados para a "nuvem" quando há conectividade. Se a rede cair, nada se perde, e a coleta e a geração de alertas continuam funcionando.

Componente (Wokwi)	Pino	Função
DHT22 (sensor 1)	GPIO 15	Temperatura corporal (°C) e umidade (%)
Botão "PULSO" (sensor 2)	GPIO 4	Cada clique representa um batimento; o firmware calcula o BPM
Chave deslizante	GPIO 27	Simula a conectividade: variável booleana `wifiConectado`
LED verde	GPIO 26	Indica que o dispositivo está online
LED vermelho	GPIO 2	Indica alerta clínico detectado na borda

Os arquivos do projeto são `sketch.ino` (código C++ comentado), `diagram.json` (circuito) e `libraries.txt` (biblioteca *DHT sensor library for ESPx*).

2. Fluxo de funcionamento

A cada iteração do `loop()`:

1. **Leitura da conectividade.** O firmware lê a chave deslizante e atualiza `wifiConectado` (`true` = online). Cada mudança de estado é registrada no Monitor Serial.
2. **Coleta periódica.** A cada 2 s (5 s em produção) o firmware lê o DHT22 e calcula o BPM. O cálculo usa a interrupção do botão: cada clique registra seu instante (com *debounce* de 250 ms, ou seja, no máximo 240 bpm), e o BPM é estimado pelos intervalos dos batimentos nos últimos 15 s: `BPM = 60000·(n−1)/(t_último − t_primeiro)`. Se não houver batimento há mais de 5 s, o BPM é 0.
3. **Processamento local (Edge).** Cada amostra é classificada sem depender da nuvem, e o resultado é gravado num *bitmask* de alertas:
   - febre: T ≥ 38 °C
   - hipotermia: T < 35 °C
   - taquicardia: BPM > 120
   - bradicardia: 0 < BPM < 50
   - sem pulso: BPM = 0
   - falha do sensor: leitura NaN

O LED vermelho acende na hora, mesmo offline. Esse é o ganho principal do Edge: **a decisão crítica não depende da internet**.

4. **Persistência local.** Toda amostra é gravada primeiro no armazenamento local, antes de qualquer tentativa de envio. Cada registro guarda número de sequência, instante, temperatura×10, umidade, BPM e alertas.
5. **Sincronização.** Se `wifiConectado == true`, o backlog é enviado para a "nuvem" via `Serial.println` (prefixo `[NUVEM]`, formato CSV `seq,t_ms,temp,umid,bpm,alerta`), em ordem cronológica. Em seguida, o armazenamento local é apagado.

3. Lógica de resiliência offline

Armazenamento local. Como o enunciado aponta, o SPIFFS é volátil nos simuladores. Por isso o código tem duas implementações, selecionadas pela constante `USE_SPIFFS`:

- `USE_SPIFFS 0` (Wokwi): uma **fila circular em RAM** (`Amostra fila[MAX_AMOSTRAS]`), com ponteiros de cabeça e cauda. O Monitor Serial funciona como meio de saída e de evidência da sincronização.
- `USE_SPIFFS 1` (ESP32 físico): cada leitura é anexada ao arquivo `/leituras.csv`. Quando a conexão volta, o arquivo é lido, enviado linha a linha e removido com `SPIFFS.remove()`. No `setup()`, o firmware conta as linhas que já existem, de modo que **dados coletados antes de um reboot também são sincronizados**.

Nenhuma amostra é removida antes de ser enviada. Na versão em RAM, o envio acontece em lotes de 20 amostras por ciclo, e o ponteiro `cauda` só avança depois do `println`. Se a rede cair no meio da sincronização, o envio é interrompido e as amostras restantes continuam na fila. Além disso, como é feito em lotes, o envio não bloqueia a coleta: o loop continua lendo os sensores entre um lote e outro.

Dimensionamento (modelo de negócio). O público-alvo é o paciente cardiológico em **monitoramento domiciliar**. As quedas típicas de rede são uma noite com o roteador desligado, um deslocamento ou uma área sem sinal. Definimos então o requisito de **4 horas de autonomia offline** com 1 amostra a cada 5 s:

> 4 h × 3.600 s ÷ 5 s = **2.880 amostras** × 16 bytes ≈ **46 kB de RAM**

Esse volume representa cerca de 14% da RAM do ESP32, o que deixa folga para Wi-Fi e TLS. Em chip real com SPIFFS (cerca de 1,4 MB úteis e cerca de 40 bytes por linha de CSV), o limite sobe para **17.280 amostras (24 h)**.

Política quando o armazenamento enche (armazenamento limitado):

1. **Decimação acima de 80% de ocupação.** As amostras normais passam a ser gravadas uma a cada duas, o que dobra a autonomia restante. **Amostras com alerta são sempre gravadas.**
2. **Buffer cheio.** É descartada a amostra **normal** mais antiga, procurada entre as 50 primeiras da fila. Os registros de alerta são preservados. Só no caso extremo, com a fila inteira de alertas, cai a mais antiga.

3. **Contadores de auditoria.** Os contadores `descartadas` e `decimadas` aparecem em todo log. Junto com o número de sequência (`seq`), eles permitem que a nuvem identifique lacunas nos dados.

A prioridade da política é clínica: perder uma leitura normal antiga tem custo baixo, e perder o registro de um evento de taquicardia tem custo alto.

4. Como testar no Wokwi

1. Crie um projeto ESP32 em https://wokwi.com/projects/new/esp32. Cole o conteúdo de `sketch.ino` e de `diagram.json`, e adicione o arquivo `libraries.txt`.
2. Inicie a simulação. Clique no botão verde **PULSO** num ritmo constante (cerca de 1 clique por segundo resulta em ≈ 60 bpm) e veja o BPM no Monitor Serial.
3. Coloque a **chave em OFF**. O LED verde apaga, as leituras continuam e o contador `buffer=` cresce.
4. Clique no DHT22 e ajuste a temperatura para 38,5 °C, ou clique rápido no botão (acima de 2 cliques/s). O LED vermelho acende mesmo offline.
5. Volte a **chave para ON**. Aparecem as linhas `[NUVEM] >>> Sincronizando backlog...`, depois as amostras em CSV e por fim `Buffer local apagado`, e o contador volta a 0.

Link do projeto no Wokwi: https://wokwi.com/projects/479719847288354817